Complexity Comparison Web App: Mini Shopping Mall

Cheolwon Park
Korea University
scott9020@korea.ac.kr

1 Web Application and Algorithm Implementation

A mini shopping mall web app was built using FastAPI (Python backend) and HTML/CSS/JavaScript (frontend). The application generates 1,050 products across 8 categories at startup, including 50 intentional duplicate names for testing purposes. Three search endpoints were implemented, each using a different algorithm of distinct time complexity:

Endpoint	Algorithm	Complexity	Implementation
`/search/id`	ID Lookup	O(1)	Python dictionary (`dict.get(id)`) hash-based direct access
`/search/name`	Name Search	O(n)	Single `for` loop iterating all 1,050 products with substring matching
`/search/duplicates`	Duplicate Detection	$O(n^2)$	Nested `for` loops comparing every pair ~550,000 comparisons

The frontend provides a search bar, three buttons for each algorithm, and displays results as product cards. Each search prominently shows the execution time (ms) measured via `time.perf_counter()`.

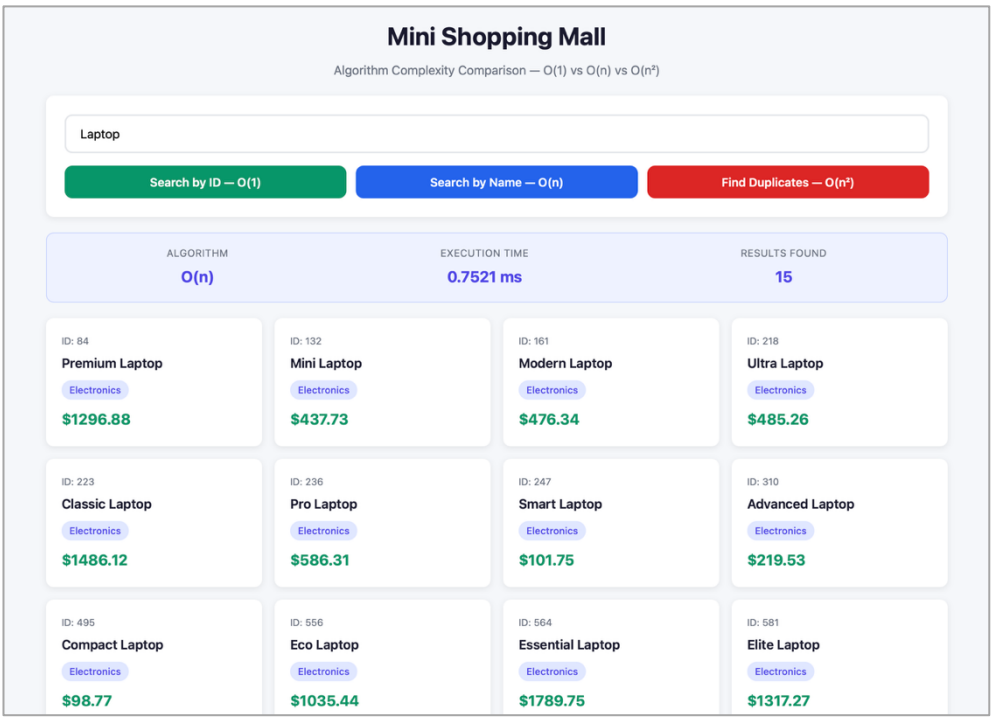

Figure 1. Shopping mall search results — Name search for "Laptop" (0.75ms, 15 results)

2 Locust Load Test Results

Load tests were conducted using Locust with three levels of concurrent users: 10, 30, and 50. All three endpoints were called using equal Locust `@task` weights to ensure fair comparison under identical load distribution.

Endpoint	Complexity	Average			95th Percentile			Max		
		10u	30u	50u	10u	30u	50u	10u	30u	50u
`/search/id`	O(1)	6.93	11.06	12.66	28	50	51	87	86	131
`/search/name`	O(n)	8.16	10.52	12.50	31	49	53	106	98	115
`/search/duplicates`	$O(n^2)$	71.39	64.64	60.99	97	110	110	116	143	150

(ms)

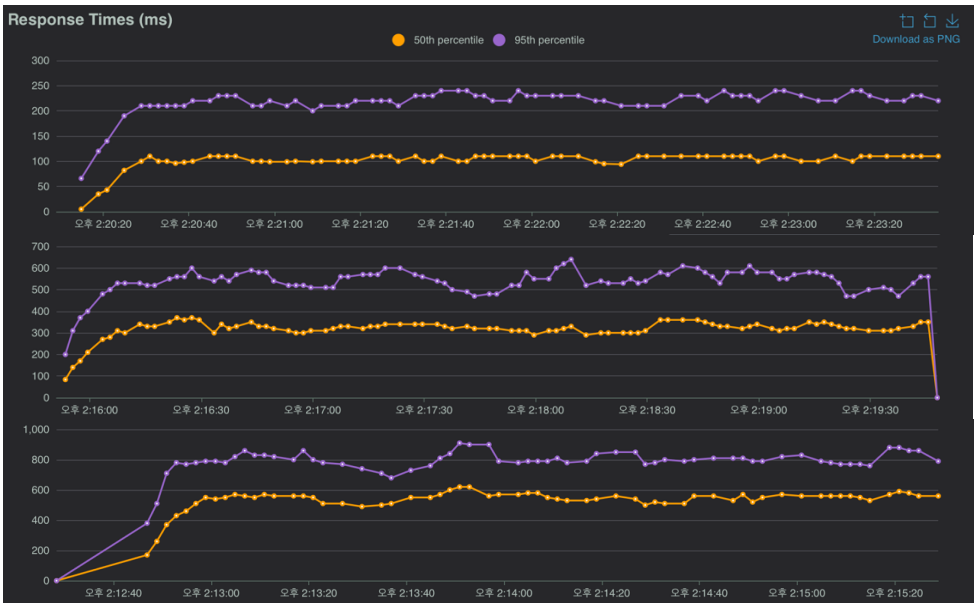

Figure 2. Locust charts of response times for 10, 30, and 50 concurrent users (top to bottom)

3 Analysis — Why does $O(n^2)$ degrade under load?

With n = 1,050 products, each algorithm performs a fundamentally different amount of work: O(1) does a single hash lookup, O(n) scans ~1,050 items, and $O(n^2)$ compares every pair, requiring $n(n-1)/2 \approx 550{,}000$ comparisons. The test data confirms this gap: $O(n^2)$ averaged 61–71 ms across all concurrency levels, while O(1) and O(n) stayed under 13 ms. Notably, O(1) and O(n) showed near-identical latency at this scale, suggesting that HTTP and framework overhead dominates when the algorithmic workload is small.

Under concurrency, the gap widens. The $O(n^2)$ endpoint is CPU-bound and blocks the server thread during its ~550,000 comparisons, forcing all other requests to queue behind it. This is why even O(1)'s max response time rose from 87 ms (10 users) to 131 ms (50 users), while $O(n^2)$'s max grew to 150 ms. Although $O(n^2)$'s average latency slightly decreased with more users, likely due to CPU cache warming across repeated calls, the 95th percentile and max latency consistently increased, confirming degradation under load. A single slow algorithm degrades the entire system, not just itself.

Replacing the nested-loop approach with a hash-based set would reduce duplicate detection from $O(n^2)$ to O(n), making it as fast as name search. Algorithm selection is a critical engineering decision that directly impacts service quality at scale.